

Section 6: LLM Model Serving

2026 Spring

[LLM Agents Foundation & Applications](#)

Student Team

20260310

We have covered

- S1: LLM Basic Alignment
- S2: LLM Alignments for Reasoning
- S3: Agent applications
- S4: LLM Data synthesis
- S5: Agent Memory
- S6: LLM model serving → This lecture!

Towards Efficient Generative Large Language Model Serving: A Survey from Algorithms to Systems (2025)

Presented by Raffi Khondaker



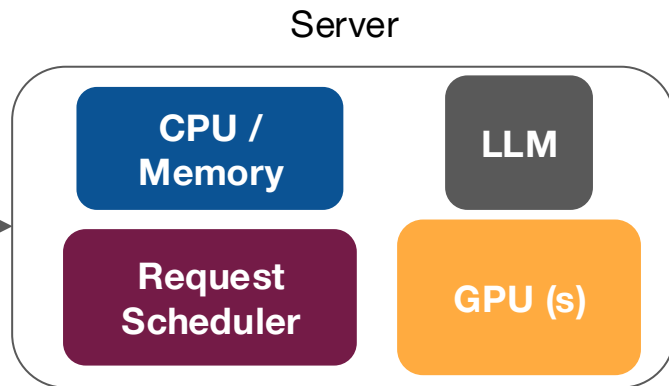
Motivation - LLM Serving

How do we deploying LLMs and respond to many user requests quickly and efficiently?

Why LLM Serving Is Difficult

- **Compute:** LLMs are large and consume significant memory and computation
- **sequential generation:** generating new tokens depends on all previously generated tokens
=> limits parallelization
- **dynamic workloads:** unpredictable requests and prompt lengths from many users at the same time

	Queue of Requests	Began Output?	Tokens to process
A	"Explain gradient descent:" "Gradient Descent is ..."	Yes	100
B	"Write a Python function for quicksort"	No	10
C	"Summarize this 1500-word document: The Gettysburg Ad..."	Yes	25
D	"Translate 'hello world' to French" "Hello in French ..."	Yes	50



How do we allocate work?

Challenges

Need high **throughput**: Number of tokens and/or requests per unit time

- Concurrent workloads: independent unpredictable requests with varying prompt lengths - needs to batch, schedule, and manage GPU resources

Need to have low **latency**: Time taken for token and/or request to be output

- Metrics: *Time To First Token (TTFT)*, *Time Between Tokens (TBT / TPOT)*
- Autoregressive generation: Cannot have full parallelization

Avoid exploding **memory** usage

- Billions to 100s of billions parameters, need 10s-100s of GiB of memory
- memory grows linearly with tokens processed (i.e caching attention values)

Sometimes need to **trade-off** between **latency**, **throughput**, and **memory**

THROUGHPUT



- Number of Data Packets arrived within one second.

LATENCY



- Time Consumed for transfer of 1 packet.

Modern LLM Serving Frameworks

Framework	Primary Goal	Release Year
FasterTransformer	Low latency	2020
FlexFlow-Serve	Low latency	2023
vLLM	High throughput	2023
FlexGen	Maximize throughput on limited GPUs	2023
HuggingFace TGI	Balanced latency + throughput	2023
DeepSpeed-Inference	Scalable distributed inference	2021
ZeRO-Inference	Memory efficiency	2022
LightLLM	High throughput	2023
MLC-LLM	Portability	2023
TensorRT-LLM	Production performance	2023

LLM serving frameworks: systems that manage and optimize the deployment of LLMs for inference for multiple concurrent user queries

- Coordinate model execution, hardware utilization, memory management, and request scheduling

These frameworks act as the runtime infrastructure layer between:

- User applications
 - a. Chatbots, search engines, coding assistants
- Hardware
 - a. GPUs, CPUs, accelerators
- Large language model itself

Modern LLM Serving Frameworks are forced to make **trade-offs** until innovations are made

LLM Serving Taxonomy

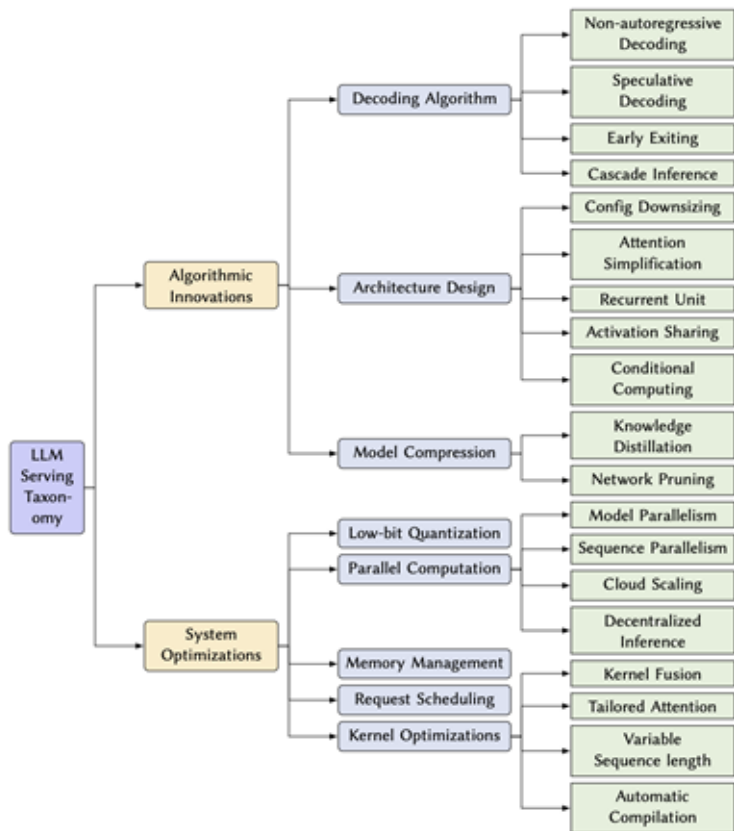


Fig. 1. Taxonomy of LLM Inference Advancements

First Paper: Survey on LLM Serving

- Approach each section as *problem* and *solution*
- Covering Breadth

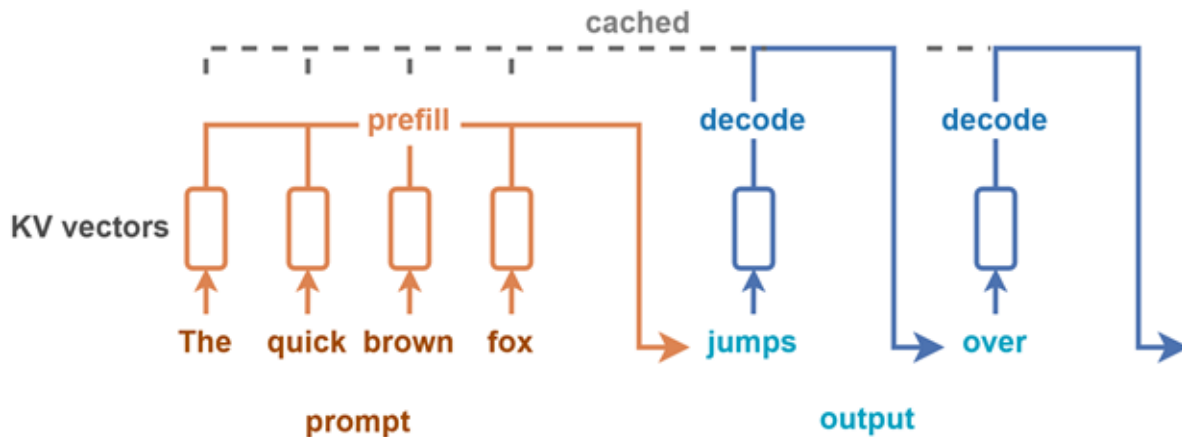
Second Paper: Sarathi-Serve

- Depth on one technique

Sequential Decoding (Problem)

LLM inference can be represented in two stages

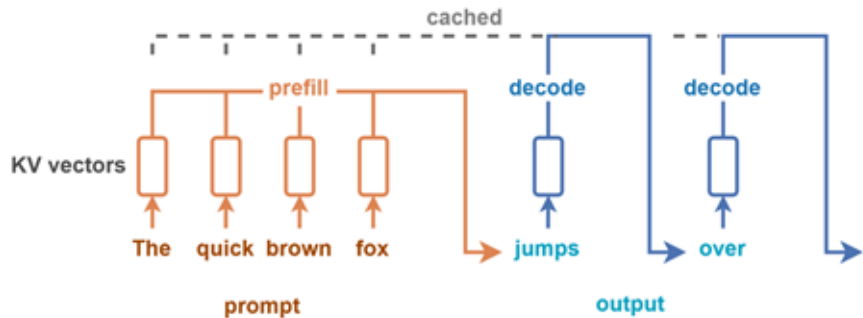
- 'Processing': Inputting tokens into LLM (1 forward pass) and doing associated computation
- **Prefill Operation**: Processing input prompt and outputting first token
 - Process each token in prompt in parallel
- **Decode Operation**: generating output tokens one at a time (autoregressive)
 - We do NOT need to recalculate values from older tokens, those are saved! (KV-Cache)



Sequential Decoding (Problem)

Prefill: Processing input prompt to outputting first token

Decode: generating output tokens one at a time
(autoregressive)



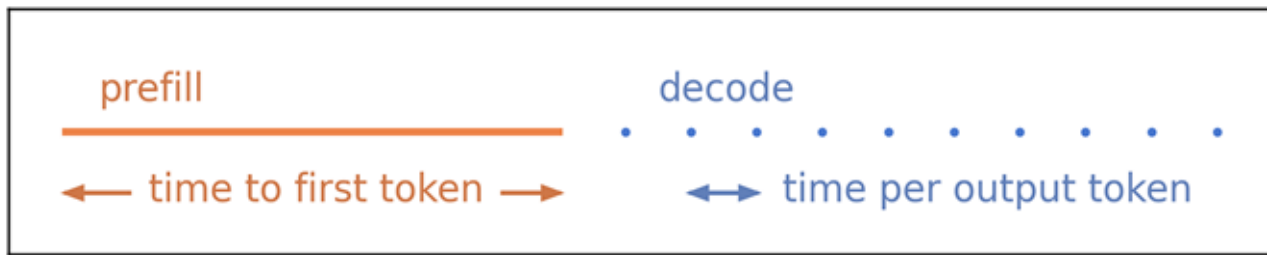
Inference looks like: Prefill (high parallelization, slow) $\Rightarrow$ Decode (less parallelization, fast) $\Rightarrow$ Decode $\Rightarrow$... $\Rightarrow$ Decode

- 1 individual decode is **fast**, but running them sequentially is **slow** (no parallelization!)
- One prefill / decode step is one **iteration** (full pass through LLM)

Problem: In decoding, each output token is conditioned on all previously generated tokens

- Forces inference to progress step-by-step, **not parallel**
- Every decoding step requires a full forward pass through the entire transformer model

Sequential Decoding (Problem)



Elapsed Time (Latency)

Latency grows linearly with decode output length

- Total response latency determined by: $\text{Latency} = \text{TTFT} + \text{TPOT} \times \text{OutputLength}$
 - TTFT (Time To First Token), TPOT (Time Per Output Token)
- Longer outputs lead to proportionally longer response time

Q: How do we decrease the latency in the entire decoding process?

Sequential Decoding: Speculative Decoding

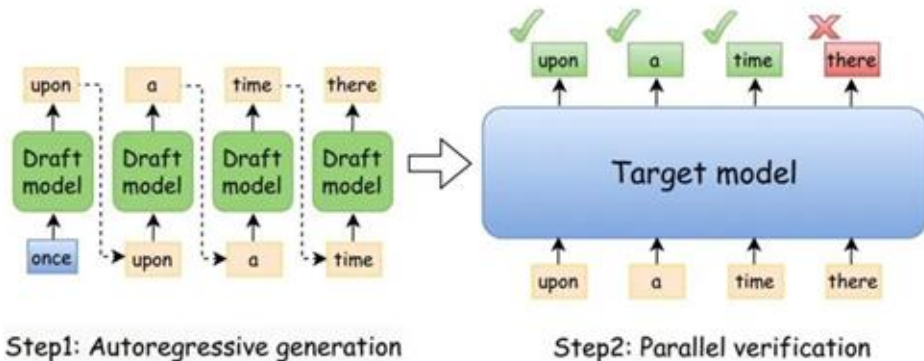
Idea: The smaller the model, the faster the forward pass, but they may lack reasoning capability of a larger model

- Have a small model predict many tokens (fast), have large model check if those are tokens large model would produce

Draft Model (Small, Fast Predictor):

- Generates multiple candidate tokens in sequence ('speculative decoding')
- Approximation of larger model

Model	Parameters	Relative Forward Pass Cost
Draft model <i>LLaMA-2 7B</i>	7B	1×
Target model <i>LLaMA-2 70B</i>	70B	~10×



Target Model (Large, Verifier): 'Original' large language model, checks which speculative predictions match the tokens that would have been produced through standard autoregressive decoding

- **Guarantees** that final outputs remain identical to normal generation

Sequential Decoding: Speculative Decoding

Draft model (q) generates candidate tokens (one token, y_t)

Large model (p) computes true conditional probabilities for candidate tokens (y_t) in 1 pass, in parallel

$p(y_t \mid y_{<t}, x)$ True Prob. of generating token y_t

$q(y_t \mid y_{<t}, x)$ Estimated Prob. of generating token y_t

Rejection Sampling: Accept token y_t with probability ~

$p(y_t) / q(y_t)$

Accept with probability $\min \left(1, \frac{p(y_t | context)}{q(y_t | context)} \right)$

- Likely to accept if draft model matches large model, or
- Draft model under-represented token compared to large model

Guaranteed this matches **Large model** probability distribution:

Probability of large model outputting y = probability of accepting y *
probability of suggesting y

- Solve for **a (acceptance probability)**
- Upper bound to 1 to ensure a probability

$$p(y_t) = a(y_t) \cdot q(y_t)$$
$$a(y_t) = \frac{p(y_t)}{q(y_t)}$$

If any candidate token is rejected, Large model predicts token - Sequence continues

Sequential Decoding: Speculative Decoding

Example: Time taken to generate 5 tokens

- Original (**250ms**):
 - Target Generates: 50ms, 50ms, 50ms, 50ms, 50ms
- Speculative (**75ms - assuming** first 5 candidates accepted):
 - Draft Generates (25ms): 5ms, 5ms, 5ms, 5ms, 5ms
 - Original Verifies (50 ms)

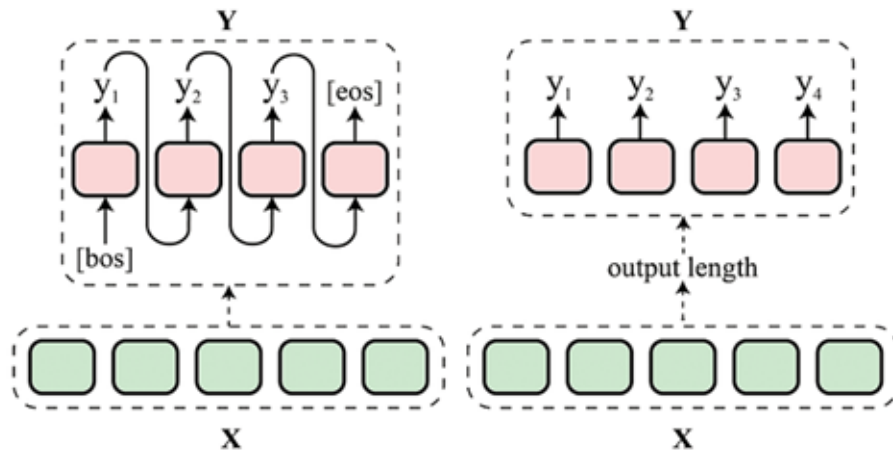
Model	Decode (1 Token) Time
Draft	5 ms
Target	50 ms

Limitation: If draft model is bad (candidate tokens always rejected), large model AND small token will generate every token

Sequential Decoding: Non-Autoregressive Techniques

Non-Autoregressive Decoding: predict multiple output tokens within a window **in parallel** instead of sequential generation

- Assumes **conditional independence between output tokens**: probability of each predicted token can be approximated without requiring previously in window



Sequential Decoding: Non-Autoregressive Techniques

Iterative refinement: Predict initial sequence of tokens predicted in parallel then progressively corrected across several decoding passes

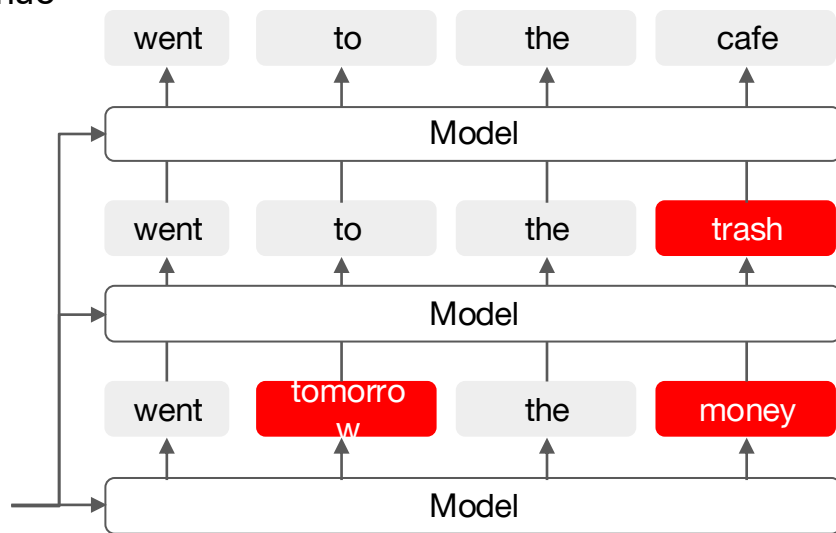
1. Output the next N tokens at once
2. Mark all tokens with low confidence (i.e softmax output beneath a threshold) for replacement
3. Re-predict all low-confidence tokens in parallel
4. Repeat until now low-confidence predictions, then continue

Limitation: degraded generation quality due to missing dependencies between neighboring tokens

 — **Low Confidence Output**

Prior Tokens

Today I went to the mall and



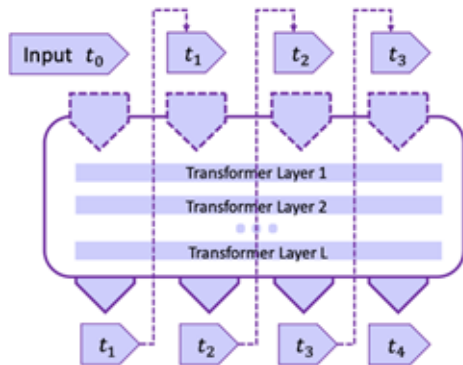
Sequential Decoding: Early Exiting

Early exiting: End forward pass early if early transformer layers can confidently predict token

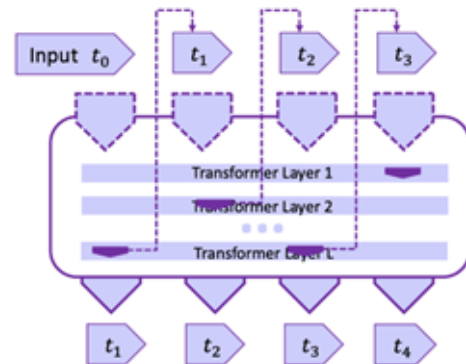
1. Classifiers to intermediate layers that attempt to predict token
2. Calculate 'confidence' in classifiers prediction
 - a. i.e entropy threshold, maximum probability threshold, margin between top tokens
3. If prediction confidence is high enough, return predicted token without executing remaining layers
 - a. Skipping future layers significantly reduced computation!

Limitation: Even if early layers are confident, final layers can still change prediction

- Early layers can easily be overconfident!



Standard Inference



Early Exiting

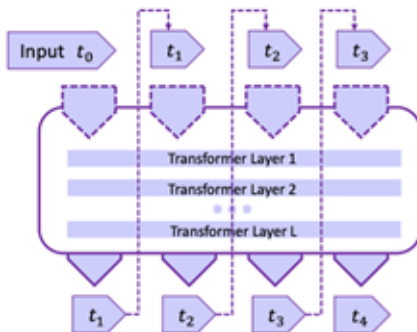
Sequential Decoding: Cascade Inference

Cascade Inference: Use multiple models of different sizes, starting from smallest, instead of one large model

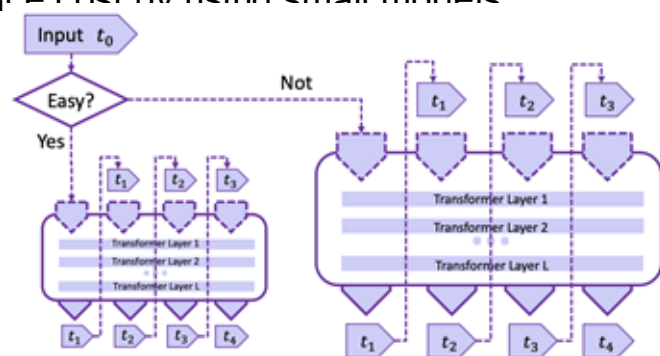
1. First run smaller, faster model that attempts to answer the query
 - a. If the model is confident in its prediction, the result is returned immediately.
2. If confidence is low, the request is passed to a larger model that has greater reasoning capability.

Assuming most queries are simple, significantly reduce inference cost by using small models

Limitation: If all queries are 'hard' EVERY model, including the largest, will end up running



Standard Inference



Cascade Inference

Summary: Sequential Decoding

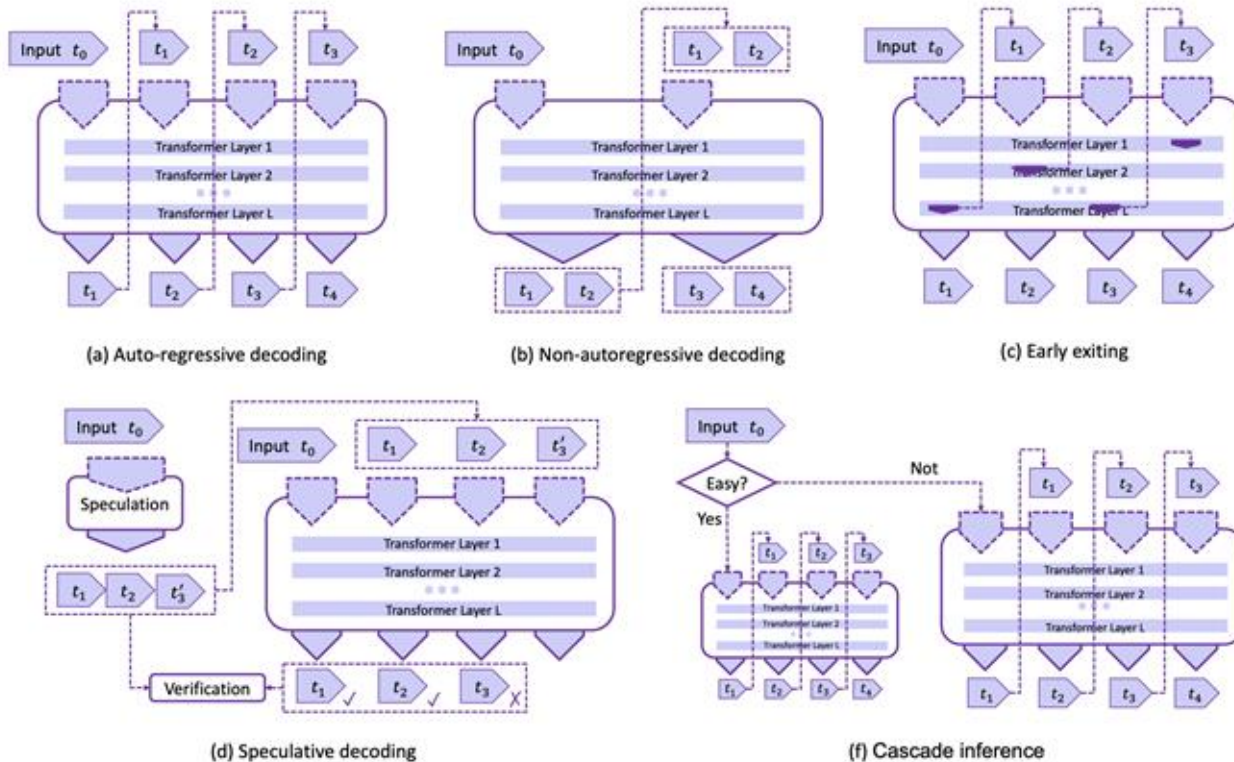


Fig. 2. Illustration of different LLM decoding algorithms

Transformer Architecture: Problem

Self-Attention: each token attends to and incorporates contextual information from all other tokens in the sequence, learns to generate:

- **complexity grows quadratically with sequence length**

Feed-Forward Network (FFN): MLP applied to each token after attention

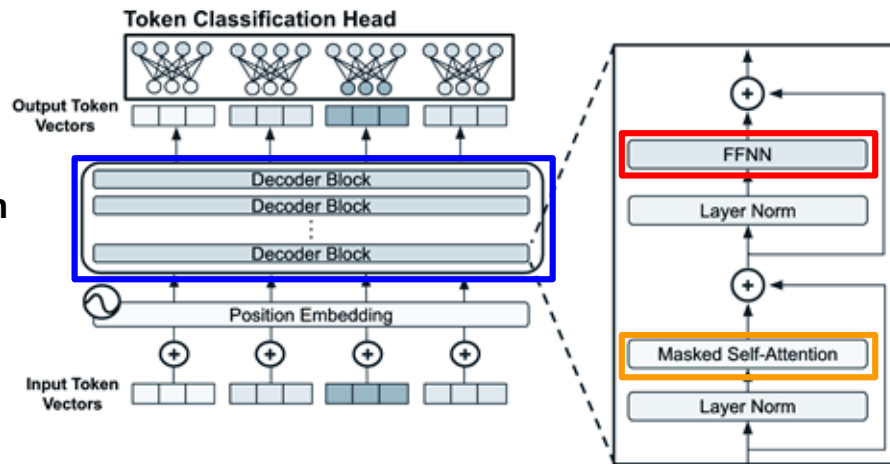
- Generally 2 layers: Linear $\Rightarrow$ Activation (ReLU or GELU) $\Rightarrow$ Linear
- **Grows linearly with sequence length, BUT has MANY parameters**

Layer Stacking in Transformers: repeated blocks of self-attention and FFNs $\Rightarrow$ refine representations at each block

- Where is time spent? Short prompt - FFN, Long prompt - attention

Modern LLMs have tens to hundreds of billions of parameters distributed across layers

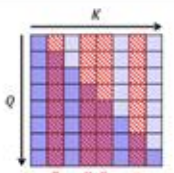
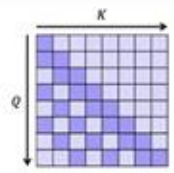
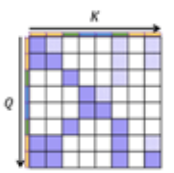
- How can we change the transformer architecture to make it less computationally expensive?



Transformer Architecture: Efficient Attention

Standard Attention: Runtime: $O(L^2d)$, Memory: $O(L^2)$, L = Sequence Length, d = dimension of KVQ

Sparse attention: each token to attend to a subset of tokens rather than the entire sequence

Attention Type	Selective	Sliding + Dilated	Hash-based
Sparse Pattern			

Sliding-window attention:

Each token attends only to nearby tokens within a fixed window, focusing on local context

- Tradeoff: Strong local context, but loses global context
- Runtime: $O(Lwd)$, Memory: $O(Lw)$, w = length of window

Dilated attention:

Each token attends to tokens at fixed intervals

- Tradeoff: Captures long-range dependence, but misses intermediates ('blind spots')
- Runtime / Memory: $O(L^2 / k)$, k = interval size

Hash-based / clustered attention:

tokens grouped into buckets or clusters, calculate attention only within groups

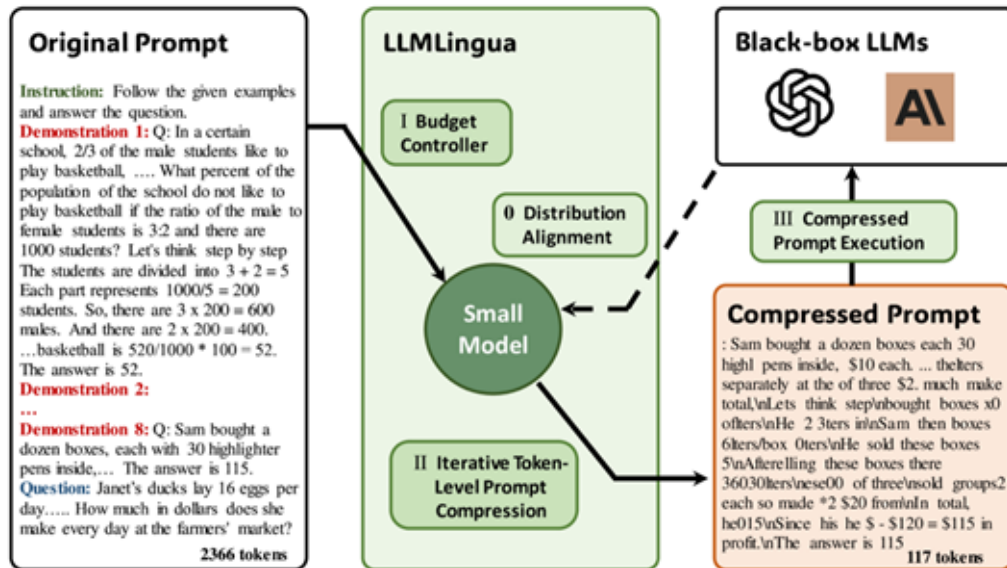
- Trade Offs: Groupings are semantic, so captures semantic relations, but highly dependent on clustering function
- Runtime: $O(L^{3/2})$, Memory: $O(L^2/b)$, b = number of buckets, each bucket is L / b long

Transformer Architecture: Efficient Attention

Context compression: reduce number of tokens in attention by summarizing or removing less important tokens from context

- **summary tokens or landmark tokens:** Incorporate earlier tokens as shortened summaries
- **semantic token pruning:** removing tokens that contribute little to future predictions using learned importance scores

Trade Offs: Inevitable loss of information, though able to preserve semantically important content



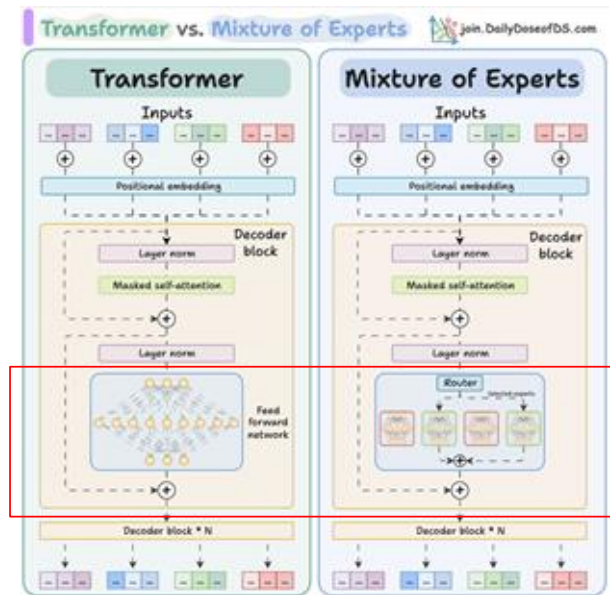
Transformer Architecture: Conditional Computation

Conditional Computation: activate a subset of model parameters during each forward pass

- Increase model capacity while lowering per-token computation

Mixture-of-Experts (MoE): partition FFN layers into smaller *expert* subnetworks, tokens directed to experts instead of one large network

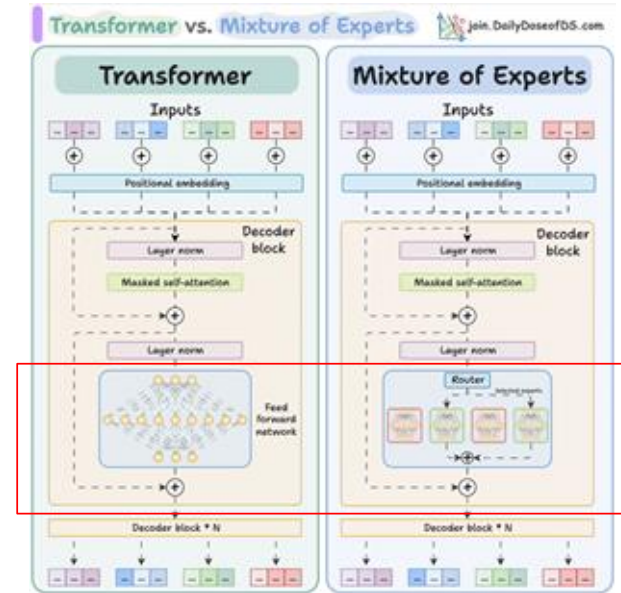
- Expert: independent FFNs specializing in a subset of input patterns or token representations
- Router (gating network): lightweight neural network, decides which experts should process token's attention/layernorm output



Transformer Architecture: Conditional Computation

MoE models can contain hundreds of billions of parameters while maintaining similar inference cost to dense Transformer models

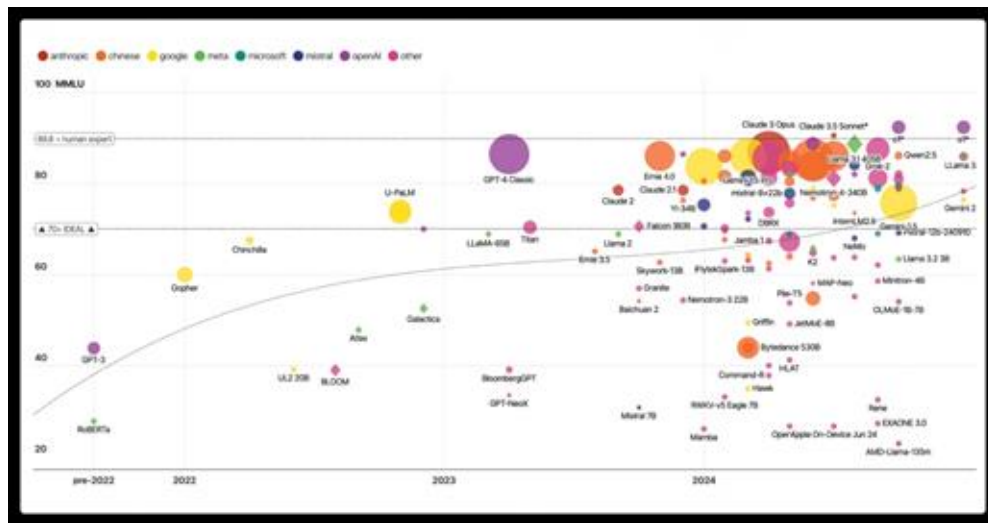
Trade-offs: Complex architecture makes managing memory, handling parallelism, and managing communication overhead difficult (*but not impossible!*)



Model Compression: Problem

Extremely Large LLMs: Billions of parameters, extremely large memory footprints and significant computational requirements during inference

- Larger parameter matrices require more memory bandwidth, more floating-point operations, and longer data transfer times between GPU memory and compute units
- Larger LLMs consume more energy, greater impact on the environment



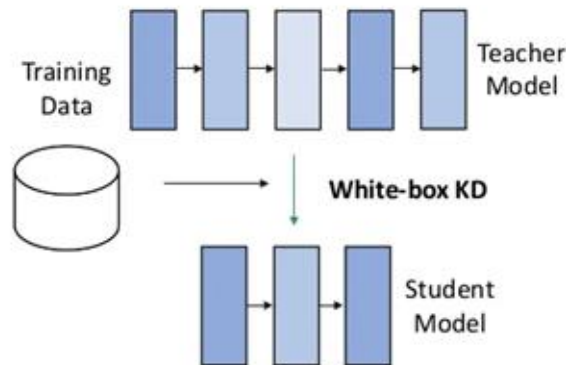
*Performance vs Time,
Relative size given by circle*

Model Compression: Knowledge Distillation

Smaller “student model” is trained to reproduce the behavior and outputs of a larger “teacher model”

White-box distillation: student has full access to the teacher model's internal parameters and intermediate activations

- Student can learn to predict teacher's hidden layer representations, attention maps, and/or intermediate logits
- **Trade-offs:** Access to richer data compared to input/output tokens and thus better transfer, but full model access is not always possible



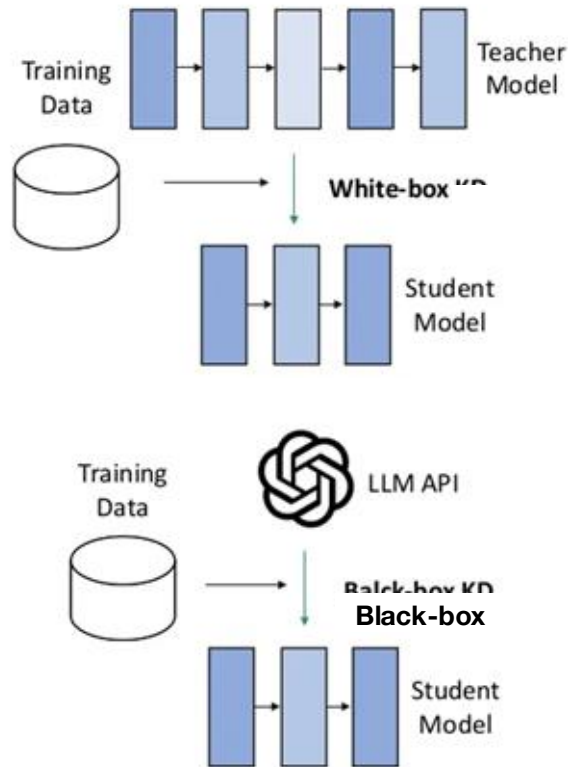
Model Compression: Knowledge Distillation

Smaller “student model” is trained to reproduce the behavior and outputs of a larger “teacher model”

White-box distillation: student has full access to the teacher model's internal parameters and intermediate activations

Black-box distillation: Student can only access teacher's outputs (i.e via API requests) - generate a ‘synthetic dataset’, with labels being the teacher's outputs

- **Trade-offs:** Possible to mimic behavior of non-open source, production-grade LLMs (i.e OpenAI's), but API requests can be expensive when generating large amounts of data
- Vicuna does this (some more examples)



Model Compression: Network Pruning

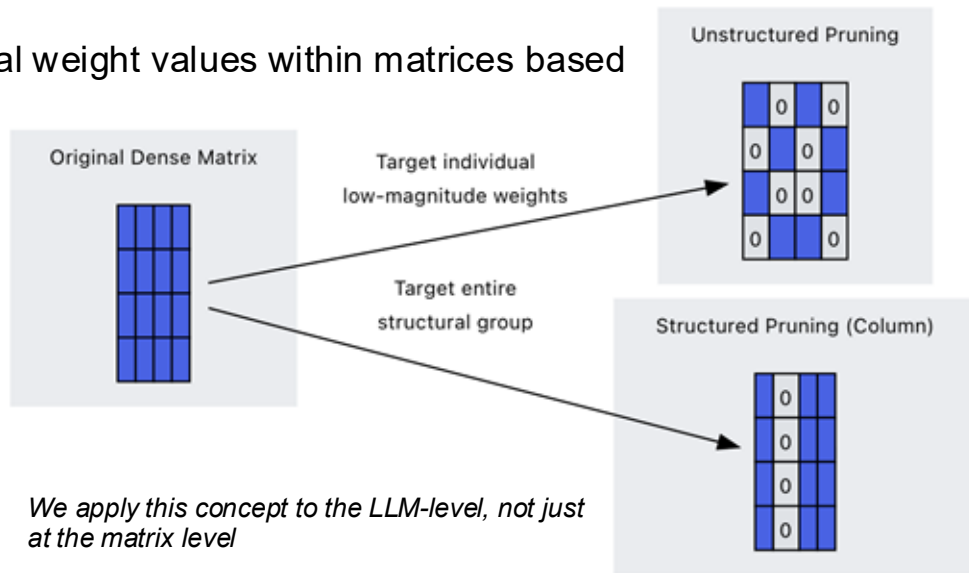
Removes parameters, neurons, attention heads, or other architectural components that contribute minimally from **pre-trained model**

Structured Pruning: removes entire units - attention heads, neurons, channels, or full matrix rows and columns

- Trade Offs: Removing redundant components directly leads to a speedup, but is not very precise

Unstructured Pruning: removes (sets to 0) individual weight values within matrices based on 'importance'

- Trade Offs: High-precision, but because all dimensions remain the same, some GPUs cannot significantly lead to a speedup



Model Compression: Network Pruning

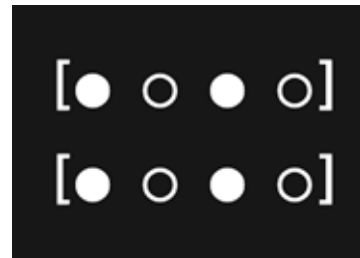
Removes parameters, neurons, attention heads, or other architectural components that contribute minimally from pre-trained model

Structured Pruning: removes entire units - attention heads, neurons, channels, or full matrix rows and columns

Unstructured Pruning: removes (sets to 0) individual weight values within matrices based on 'importance'

Semi-Structured Sparsity (compromise): fixed number of weights are removed within small groups, compatible with modern GPUs with dedicated sparse tensor cores

- Example 2:4 Sparsity: removing two weights out of every four within small blocks of a matrix, GPU treats as a 2x2 matrix, not a 4x4
⇒ Massive speedup!
- Trade-offs: Removing every N (i.e 2) columns is a difficult, fairly arbitrary rule to work with



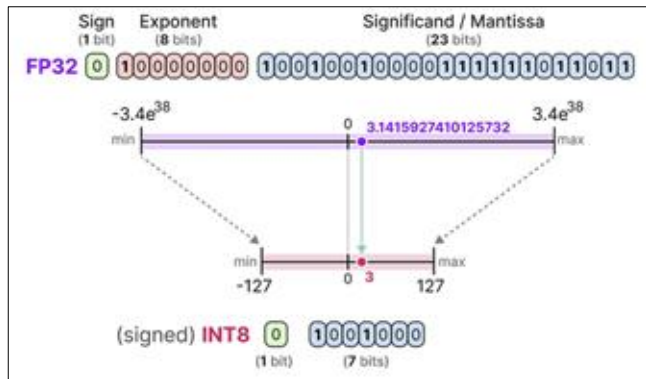
Model Compression: Quantization

Quantization: Model weights stored with lower numerical precision, typically converting 32-bit floating point values into lower precision formats such as 16-bit, 8-bit, or 4-bit integer representations

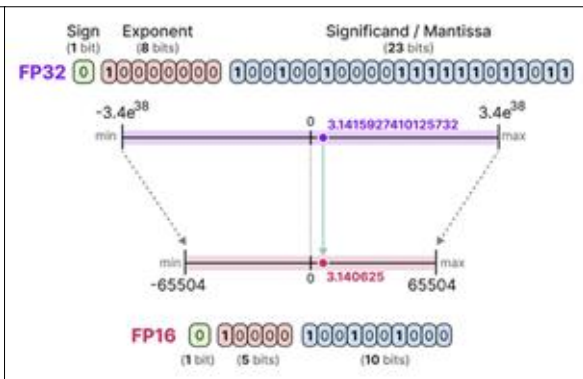
- Lower-precision dramatically reduce storage requirements and memory bandwidth demands during matrix multiplication operations

Common quantization schemes in LLMs - Less bits lead to more efficient systems at the cost of performance

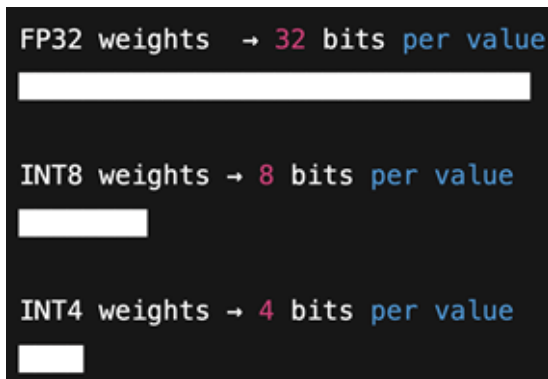
- *INT8* (store parameters as 8-bit integers), *INT4* (extremely aggressive, potential information loss)
- *W8A16*: weights are stored in INT8 while activations remain in 16-bit floating point precision



FP32 $\Rightarrow$ INT8 Diagram



FP32 $\Rightarrow$ FP16 Diagram



FP32 weights $\rightarrow$ 32 bits per value

INT8 weights $\rightarrow$ 8 bits per value

INT4 weights $\rightarrow$ 4 bits per value

Model Compression: Quantization

Post-Training Quantization (PTQ): compresses a trained model after training has finished

Quantization-Aware Training (QAT): incorporates quantization during training so the model learns to compensate for reduced numerical precision

Axis	QAT	PTQ
End Accuracy	Higher	Lower
Training Cost	Higher	Lower
Deployment Cost	Higher	Lower
Dev efforts of Implementation	Higher	Lower
Application goals	Accuracy is critical.	Resource constrained deployment surfaces / large models

Quantization allows large models to fit within limited GPU memory and significantly increases serving capacity per GPU

- Reducing precision from FP16 to INT8 approximately halves parameter memory usage
- INT4 representations can reduce model size by nearly 4× relative to FP16
- Lower memory consumption allows larger batch sizes and more concurrent inference requests

Parallel Inference: Problem

Modern LLM parameter counts can exceed memory capacity of a single GPU **during inference**

Memory needed not just to load model weights - i.e intermediate activations and KV cache

- Reminder: KV cache memory increasing linearly with sequence length
- meaning longer prompts and outputs significantly expand memory consumption

Large Memory usage limits throughput

- Directly limits batch size - controls how many requests can be processed in parallel on a single GPU
- **Consequence:** each GPU device runs **at most a single request at a time**

GPU	Memory (GB)
NVIDIA T4	16 GB
NVIDIA V100	16–32 GB
NVIDIA A100	40 / 80 GB
NVIDIA H100	80 GB
NVIDIA H200	141 GB
AMD MI300X	192 GB

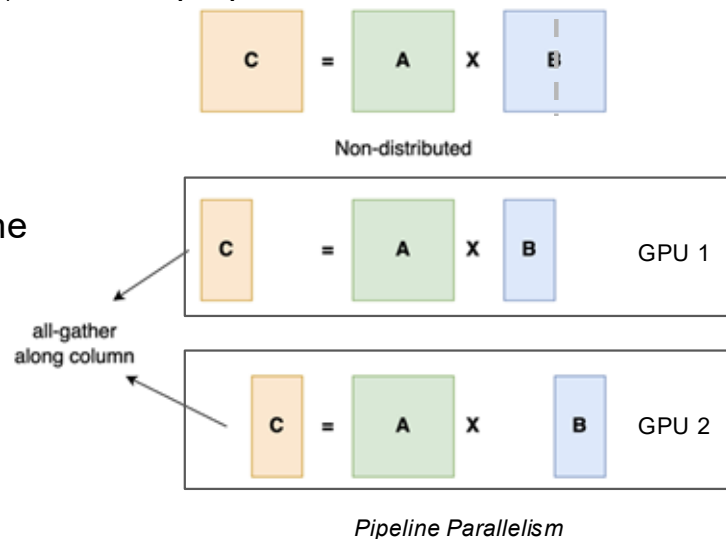
Model	FP16 Memory	Fit on Single GPU?
LLaMA-7B	~14 GB	Yes (most GPUs)
Mistral-7B	~14 GB	Yes
LLaMA-13B	~26 GB	Only ≥24GB GPUs
LLaMA-30B	~60 GB	Only A100-80GB / H100
LLaMA-65B	~130 GB	No
GPT-3	~350 GB	No
GPT-4 (est.)	~2 TB+	Impossible
DeepSeek-V3 (MoE)	~74 GB active	with optimization

Parallel Inference: Model Parallelism

Distributing LLM computation across multiple hardware devices

Model Parallelism: Partition parameters and computation across multiple GPUs so that each device stores and computes only a subset of the model

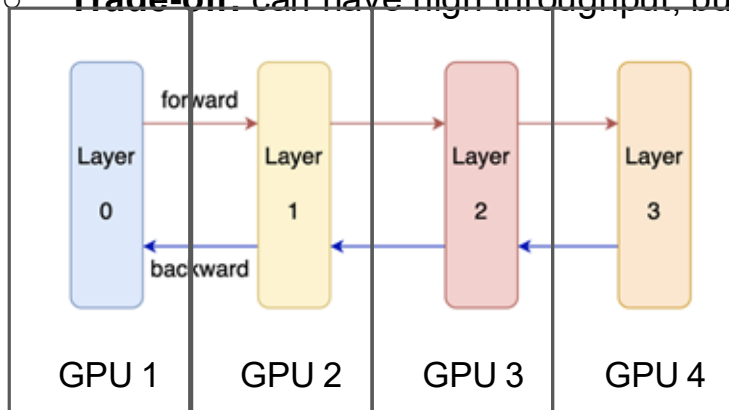
- **Tensor parallelism:** Splitting large matrix multiplications (e.g., attention projections or feed-forward layers) across GPUs
 - each device computes partial outputs that are later combined (i.e. *all-gather* operation)
 - **Trade-off:** Can have low latency, but the more GPUs, the more communication overhead at the combination step



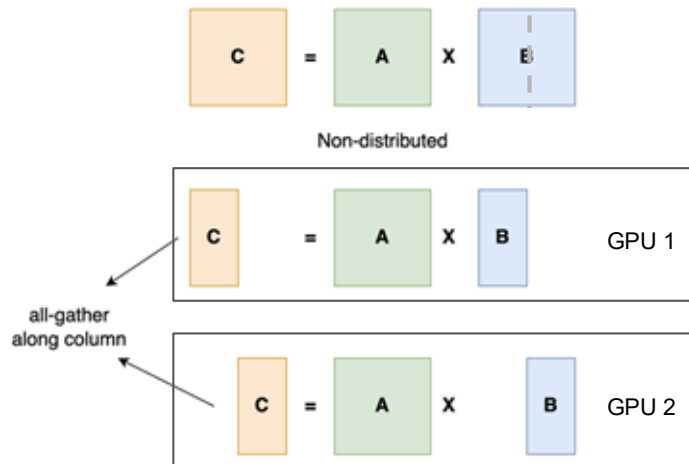
Parallel Inference: Model Parallelism

Model Parallelism: Partition parameters and computation across multiple GPUs

- **Tensor parallelism:** Splitting large matrix multiplications (e.g., attention projections or feed-forward layers) across GPUs
- **Pipeline parallelism:** Splitting layers of transformer across devices where each GPU executes a subset of layers and passes intermediate activations to the next stage
- **Trade-off:** can have high throughput, but sy



Tensor Parallelism

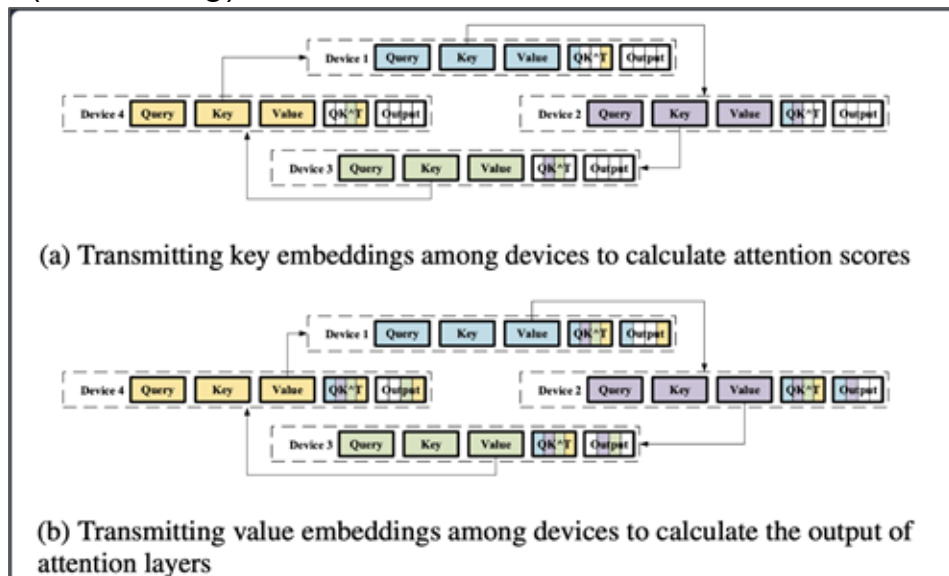


Pipeline Parallelism

Parallel Inference: Sequence Parallelism

Sequence Parallelism: Distribute groups of contiguous tokens across GPUs

- Challenge: Attention needs compare K, Q, Vs from all tokens
- Solution: GPUs calculate K, Q, Vs, GPUs share (i.e in a ring) to calculate attention scores
- **Trade-off:** communication overhead between GPUs introduced **interconnect** bandwidth is insufficient
- Interconnects: networks used to exchange activations and attention outputs
 - NVLink: GPU to GPU
 - PCIe: GPU to CPU



Example of sequence parallelism: Ring Attention

In practice, many systems use a combination of model (pipeline, tensor) and sequence parallelism

Memory Management: Problem

Key–Value (KV) Cache: memory consumption increases linearly with generated tokens

Contiguous KV cache over-allocation (Naive Approach): reserve large continuous block of GPU memory each request based on a maximum possible sequence length

- Memory block reserved even if the actual prompt or output length is much smaller



Memory fragmentation: free GPU memory becomes divided into many small non-contiguous regions

- Deallocated memory leaves “holes”, preventing allocation of large memory blocks even when total free memory is sufficient



Fragmentation and over-allocation together waste large portions of GPU memory, limiting batch size and significantly reducing throughput of LLM serving systems

Memory Management: Paged Attention

Paged Attention (introduced in the vLLM serving system): divides *KV cache* into smaller fixed-size blocks called *pages*, allocate memory only when new tokens are generated

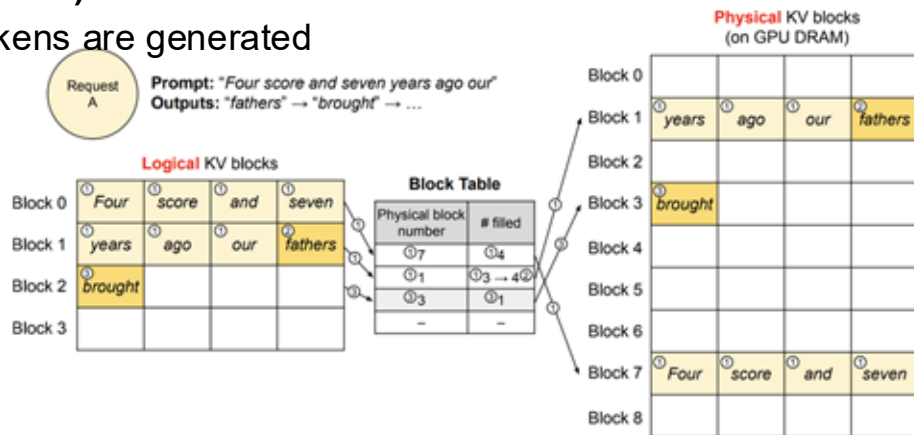
- **Pages (memory blocks):** fixed-size segments of GPU memory that store a small number of tokens' KV states

1. Break prompt into 'blocks' of tokens (i.e 16 tokens, or 1 token at a time)
2. The program sees '*N*' blocks of contiguous memory (Logical Blocks)

- a. each block maps to non-contiguous physical memory (Physical Blocks)
- b. Program 'sees' contiguous blocks, but physical memory **does not** have to be contiguous!

Result:

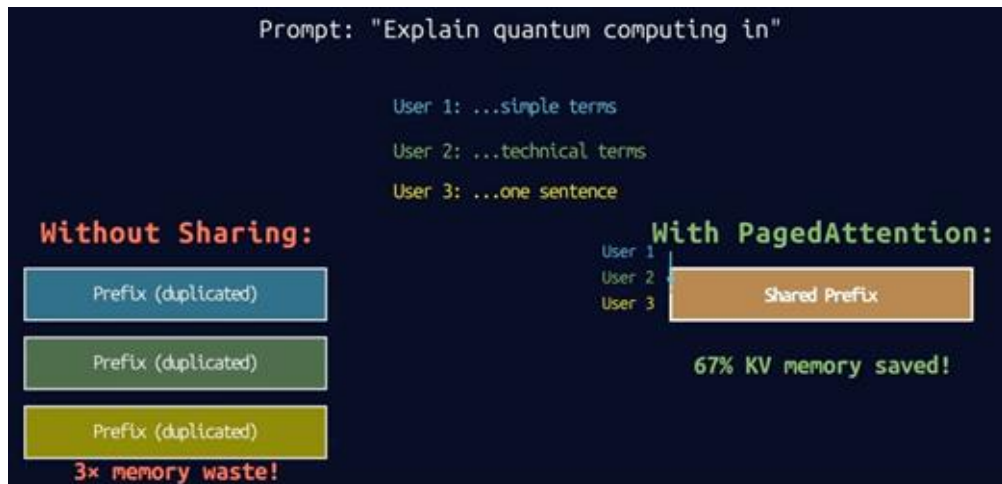
- allocate blocks of physically memory as needed,
- de-allocated physical memory can be easily filled in



Memory Management: Paged Attention

Shared memory: Prompts that have the same 'prefix' can reference the same blocks of memory

- Large reduction in redundancy!



PagedAttention solves:

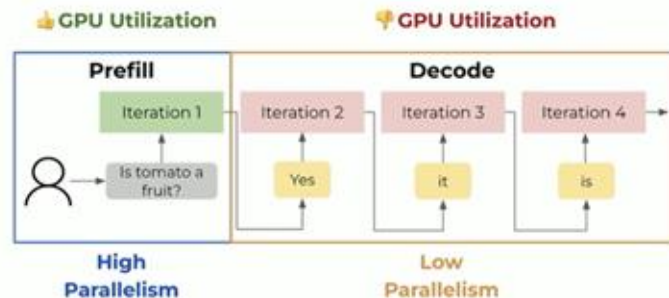
- Fragmentation - allocated / de-allocated blocks are always the same size
- Over-allocation - we allocate new blocks as more blocks of tokens come in

Allow for larger batch-sizes $\Rightarrow$ Higher throughput

Request Scheduling: Problem

Requests-level Scheduling (naive): one request occupies GPU resources until full output generation finishes

- *Remember:* Inference is split into
 - **prefill** (processing prompt, high GPU usage)
 - **decode** (generating 1 token at a time, low GPU usage)
- Handling one request at a time prevents other requests from sharing computation during intermediate decoding iterations



Static Batching (naive): Let's say we *do* handle multiple decode-requests in parallel - we wait until all requests in batch complete

- Results in lower GPU utilization as requests accumulate

3 Empty slots in batch!

Legend:

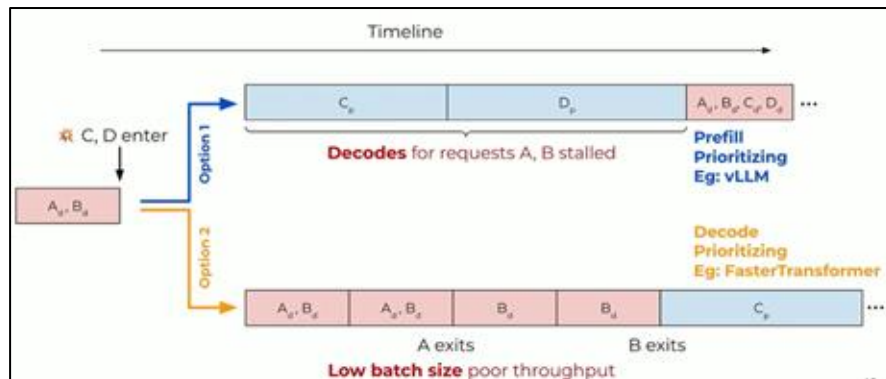
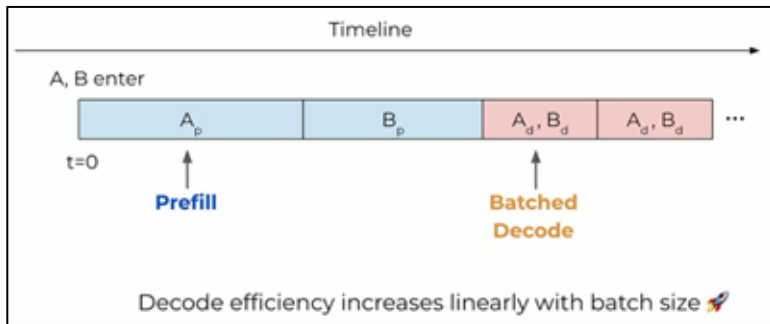
- Yellow: prompt token
- Blue: generated token
- Red: end-of-sequence token

[illegible][illegible]

Request Scheduling: Iteration-level Scheduling

Iteration-level scheduling: Split each request into prefills and decodes, and decide how to best (1) batch decodes and (2) order prefills

- **Prioritize decodes:** Individual requests complete faster (low latency), but less GPU utilization (low throughput)
- **Prioritize prefills:** Decodes are paused when new prefills come in (high / variable latency), but maximize GPU usage (high throughput)



Solution: Sarathi-Serve (More on this later!)

Request Scheduling: Iteration-level Scheduling

Continuous batching: update batches after each decoding iteration, inserting new decode requests when one completes, thereby preventing GPU idle time

Addition considerations in scheduling:

- Prompt length, order of arrival
- Service Level Objective (SLO): acceptable latency targets for user-facing inference tasks such as chatbot responses or API completion requests

Legend:

- Yellow: prompt token
- Blue: generated token
- Red: end-of-sequence token

Static
Batching

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	END		
S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	END			
S_4	S_4	S_4	S_4	S_4	S_4	END	

Continuous
Batching

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	END	S_6	S_6
S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	END	S_5	S_5	S_5
S_4	S_4	S_4	S_4	S_4	S_4	END	S_7

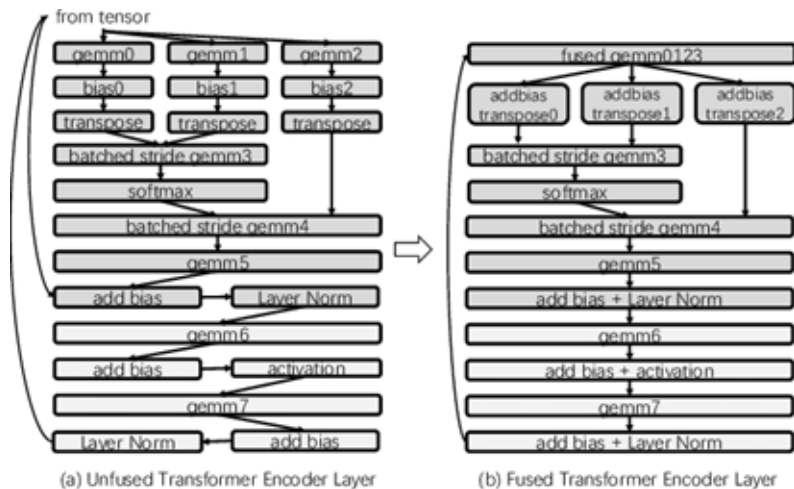
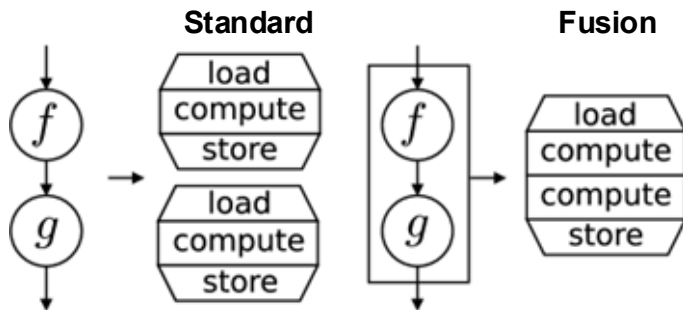
Kernel Fusion

Kernel Fusion: combining multiple GPU operations into a single kernel to reduce kernel launch overhead and **minimize repeated reads and writes to GPU memory**

- GPU kernel: a small program executed in parallel across thousands of GPU threads (i.e matmuls, softmax, normalizations)
- Kernel launch overhead: delay each time GPU scheduler dispatches a new kernel execution

In Transformer inference, though would be to fuse an entire **block** into one kernel:

- linear projection for queries, keys, values , bias addition and residual connections, layer normalization and activation functions

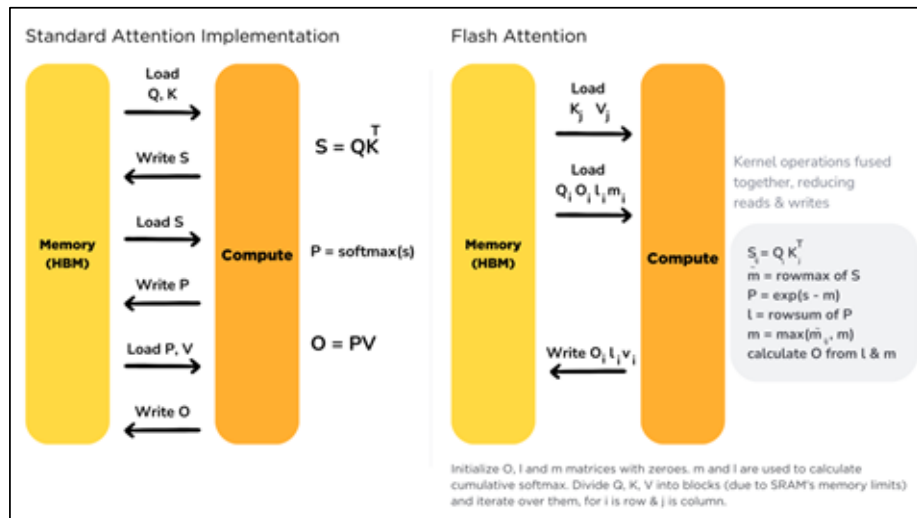
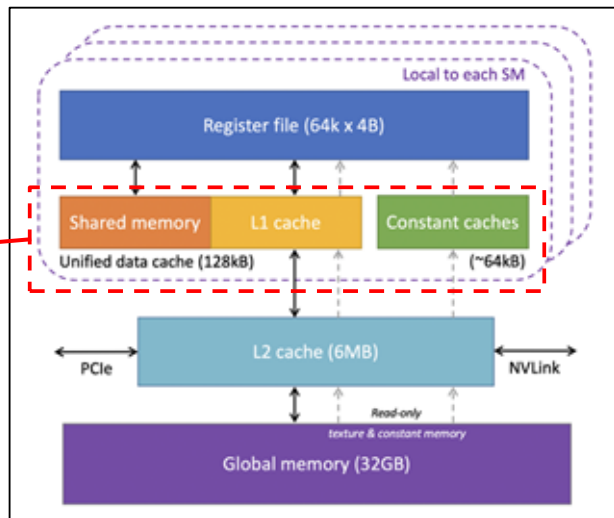


Kernel Fusion

Tailored Attention Kernels: specialized GPU implementations of attention designed to maximize thread utilization and minimize memory bandwidth consumption during both prompt processing and token generation

- FlashAttention: reduces memory bandwidth usage by computing attention blocks in shared memory rather than repeatedly reading from global memory

Maximize work done here!
~10-100x faster than global memory



Modern LLM Serving Frameworks

Review: LLM serving frameworks are software infrastructure responsible for executing inference requests from users, managing model execution pipelines, scheduling GPU workloads, and optimizing memory and compute utilization across hardware resources

Representative LLM serving systems trade-off between minimizing latency, maximizing throughput, or large-scale distributed inference

- **vLLM:** Introduced paged attention
- **TensorRT-LLM:** from NVIDIA, optimized CUDA kernels (fusion), FlashAttention, and Triton serving infrastructure to maximize GPU hardware efficiency
- **DeepSpeed-Inference:** distributed inference engine supporting tensor parallelism, kernel fusion, and CPU/NVMe memory offloading to scale large models across multi-node clusters
- **LightLLM and FlexGen:** frameworks emphasizing memory efficiency through token-level KV cache management and hierarchical memory offloading between GPU, CPU, and storage

	Optimizations						
Framework	Algorithmic	Memory	Scheduling	Kernel	Parallelism	Primary Goal	Brief Explanation
FasterTransformer	✗	✗	✗	✓	✓	Low latency	Maximize GPU efficiency for single requests
FlexFlow-Serve	✓	✗	✓	✓	✓	Low latency	Speculative + parallelization to reduce decode latency
vLLM	✗	✓	✓	✓	✓	High throughput	Introduces paged attention allowing larger batch sizes
FlexGen	✗	✓	✗	✗	✓	Maximize throughput on limited GPUs	Uses CPU + disk offloading to run large models on smaller GPU memory
HuggingFace TGI	✗	✓	✓	✓	✓	Balanced latency + throughput	Combines FlashAttention, paged attention, and continuous batching for scalable production inference
DeepSpeed-Inference	✗	✗	✗	✓	✓	Scalable distributed inference	Kernel injection + tensor parallelism to scale LLM inference across GPUs
ZeRO-Inference	✗	✓	✗	✓	✓	Memory efficiency	store model parameters outside GPU memory, saving memory
LightLLM	✗	✓	✓	✓	✓	High throughput	Token-level KV cache management and optimized attention kernels
MLC-LLM	✗	✓	✗	✓	✓	Portability	Deploy LLMs across diverse hardware
TensorRT-LLM	✗	✓	✓	✓	✓	Production performance	NVIDIA optimized stack combining FlashAttention, paged attention, and distributed inference

Conclusion + Future Work

Key bottlenecks in modern LLM serving systems arise from the interaction between sequential token generation, large model parameter sizes, and rapidly growing attention contexts

Efficiency gains emerge when algorithmic improvements and system engineering are combined

- Workload-aware optimization: design philosophy where serving systems dynamically adapt batching, scheduling, and memory allocation strategies based on request patterns and context lengths

Future research directions in LLM serving focus on scaling inference to larger models, longer contexts, and more heterogeneous computing environments

- **Hardware-software co-design:** joint development of model architectures and accelerator hardware to better exploit specialized compute units and memory hierarchies
- **Distributed inference architectures:** scalable serving systems capable of running large models across clusters of GPUs or decentralized networks while maintaining reliability and low latency

Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve (2024)

Presented by Raffi Khondaker



Motivation: LLM Scheduling

LLM inference serving must optimize **two conflicting objectives**:

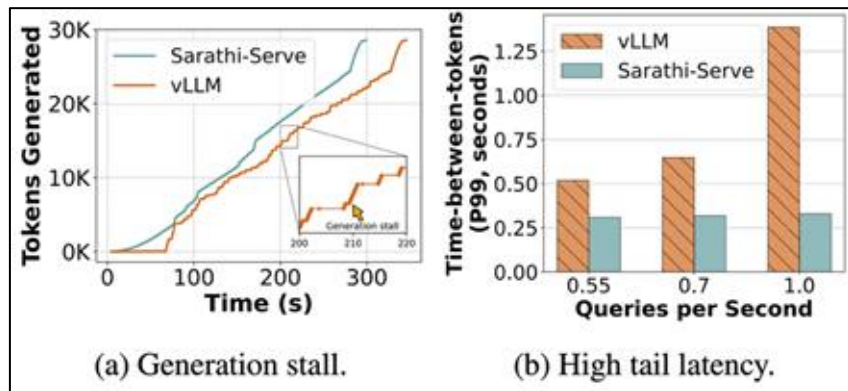
Throughput: number of requests per unit time by the serving system

Latency: responsiveness experienced by the user during text generation

- **TBT (Time-Between-Tokens)**: delay between successive generated tokens during decoding
 - Tend to be variable with 'flat' regions (generation forced to pause), Increase with higher throughput
- **P99 TBT**: 99th Percentile (1% highest delay) TBTs - Worst-case increases with throughput

Reminder: Split inference into two steps - Prefill (Processing Prompt), Decode (Generating a token)

- Iteration: One forward pass through the model: one batched decode or batched prefill in an iteration

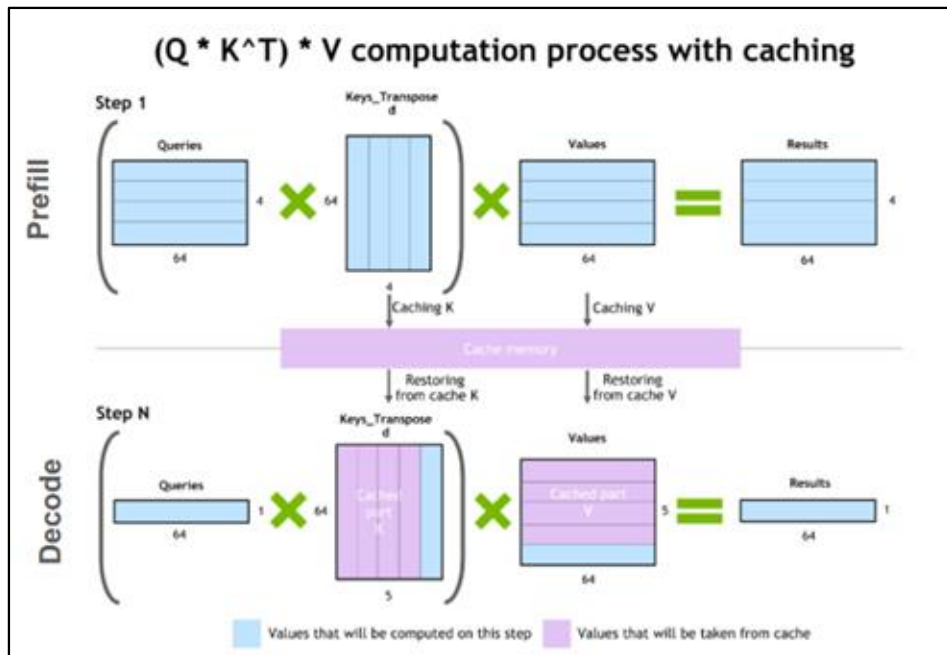


Background: Prefill

Most computation happens in QKV Projections, Attention Layers and MLP Layers

Prefill (Processing prompt of sequence length L): Attention scales quadratically with L , MLP and QKV scale linearly with L

- Most computation are matrix multiplications
⇒ Lots of highly parallelizable compute ⇒ lots of GPU throughput
- **Compute bound:** We can still get *faster* with efficient algorithms and more parallelization



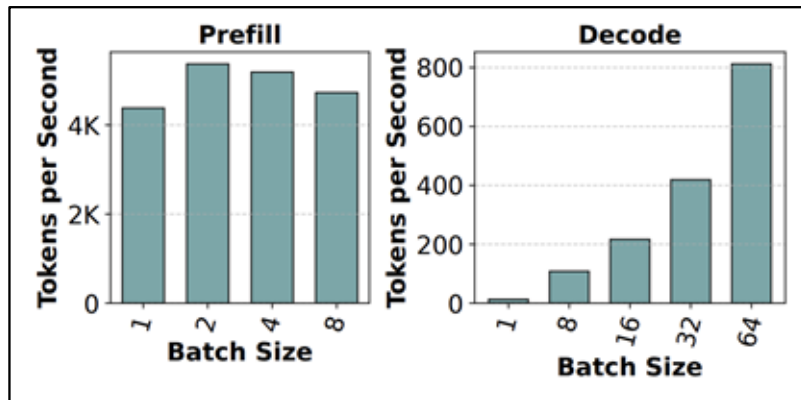
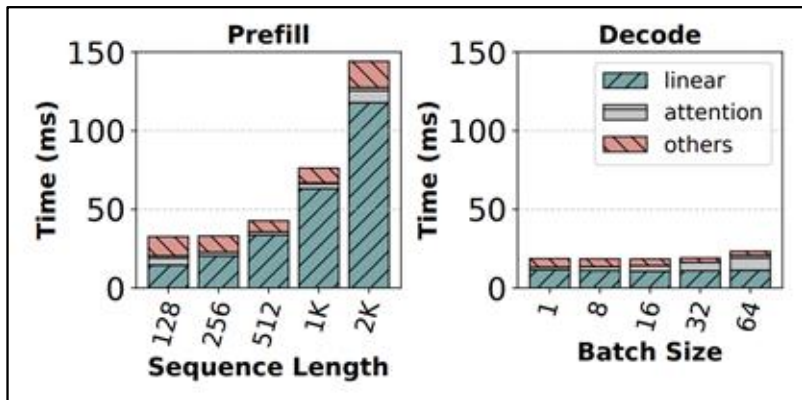
Background: Decode

Decode (Sequence of 1): Constant with Attention and MLP computations

- Does NOT need to recompute past attention scores: KV Cache stores previous keys and attention output
- Less compute that's somewhat parallelizable $\Rightarrow$ Low latency (little compute), but low throughput
 - **Memory bound:** Can only get faster with less and faster memory transfers

Batching improves decode throughput dramatically but has minimal impact on prefill throughput

- prefill already saturates compute resources even with small batch sizes



Related Work: Decode vs Prefill Prioritizing Schedulers

Decode-prioritizing schedulers: delay new prefills until ongoing decodes complete

- Used in early inference frameworks: FasterTransformer and Triton Inference Server

Pro: new prompt processing does not interrupt existing decoding iterations $\Rightarrow$ Low latency

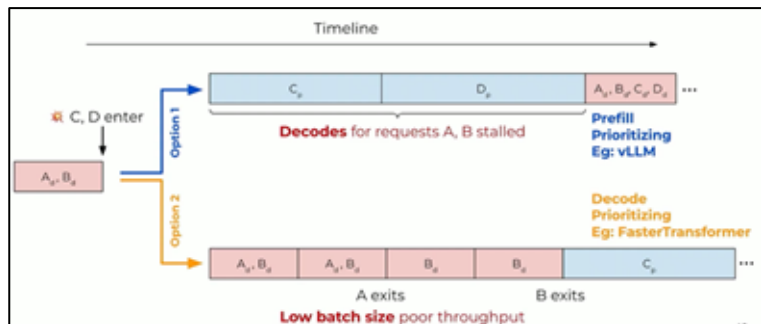
Con: Throughput suffers because decode batch size shrinks as requests finish

Prefill-Prioritizing Schedulers: prioritizing the execution of prompt processing (prefill) operations before continuing decode iterations for existing requests

- Used in vllm, Orca, TensorRT-LLM

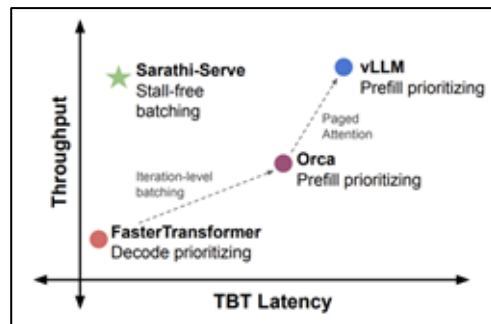
Pro: Batching a few prefills leads to maximal GPU usage and thus throughput

Con: token generation of already running requests is delayed by long prefill computations (**generation stalls**)



Forced to trade-off between latency (TBT) and Throughput

Solution: Sarathi-Serve



Sarathi-Serve: Chunked Prefills

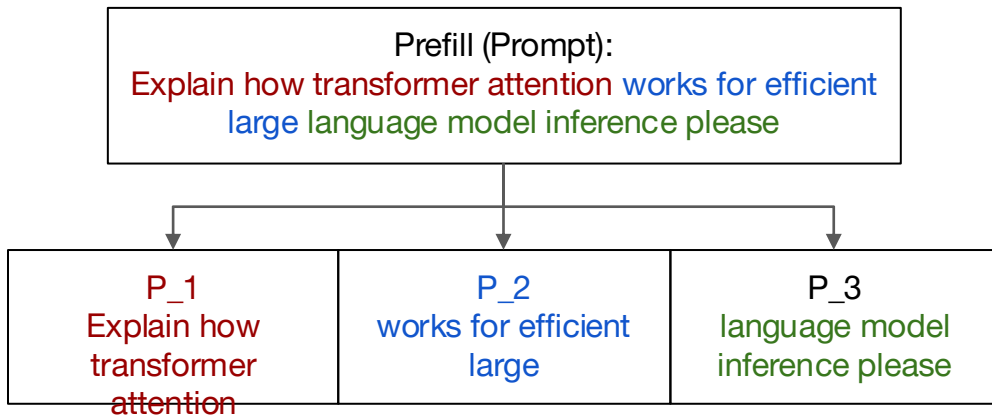
Problem: Large Prefill Iterations Stall Decode Execution

Solution: **Chunked Prefills** - divides a single prompt prefill into multiple smaller compute-balanced segments

- Prefill computation distributed across multiple iterations
- prompt tokens gradually incorporated into KV-cache over multiple scheduling cycles

Result: Bounded Iteration Latency and Stall-Free Scheduling

- Shorter and more predictable iteration runtimes: compute per iteration capped by chunk size rather than prompt length



	k0	k1	k2	k3								
q0	1	-	-	-								
q1	1	1	-	-								
q2	1	1	1	-								
q3	1	1	1	1								
attention mask during first chunk prefill												
	k0	k1	k2	k3	k4	k5	k6	k7				
q4	1	1	1	1	1	-	-	-				
q5	1	1	1	1	1	1	-	-				
q6	1	1	1	1	1	1	1	-				
q7	1	1	1	1	1	1	1	1				
attention mask during second chunk prefill												
	k0	k1	k2	k3	k4	k5	k6	k7	k8	k9	k10	k11
q8	1	1	1	1	1	1	1	1	1	-	-	-
q9	1	1	1	1	1	1	1	1	1	1	-	-
q10	1	1	1	1	1	1	1	1	1	1	1	-
q11	1	1	1	1	1	1	1	1	1	1	1	1
attention mask during third chunk prefill												

Sarathi-Serve: Stall-Free Scheduling

Stall-free scheduling: scheduling strategy that allows new requests to enter a running batch without pausing ongoing decode iterations

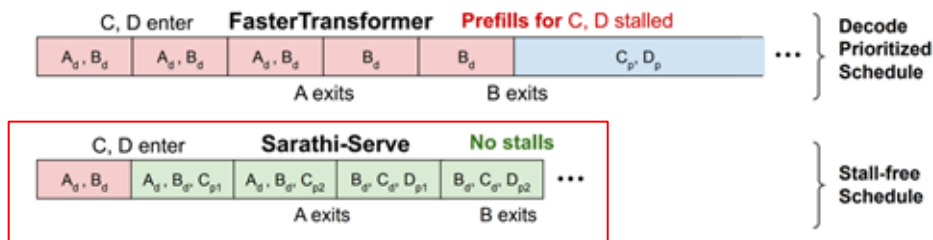
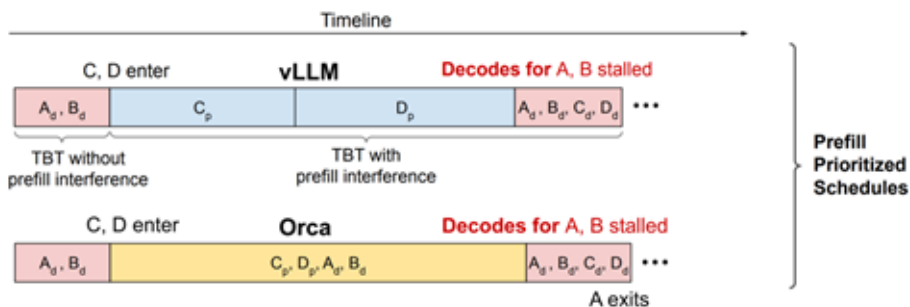
- Sarathi constructs hybrid batches that combine decode tokens with bounded-size prefill chunks

Scheduler ensures uniform batch compute cost across iterations

- compute workload per iteration bounded by a fixed chunk size

Result: continuous decoding with high throughput

- ongoing requests continue generating tokens without interruption
- GPU utilization improved by maintaining large batch sizes



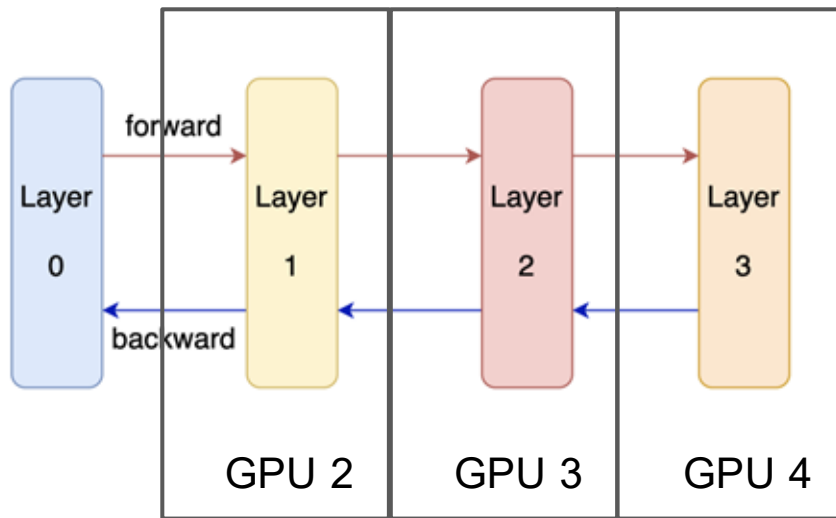
Improvements to Parallelism: Tensor

Tensor Parallelism (TP): each GPU computes partial results of matrix multiplications before combining

- Limitation: communication overhead - requires synchronization after each layer

Sarathi-Serve: balances compute across decode iterations, but does not eliminate communication bottleneck

- Modest improvement to TP



Tensor Parallelism

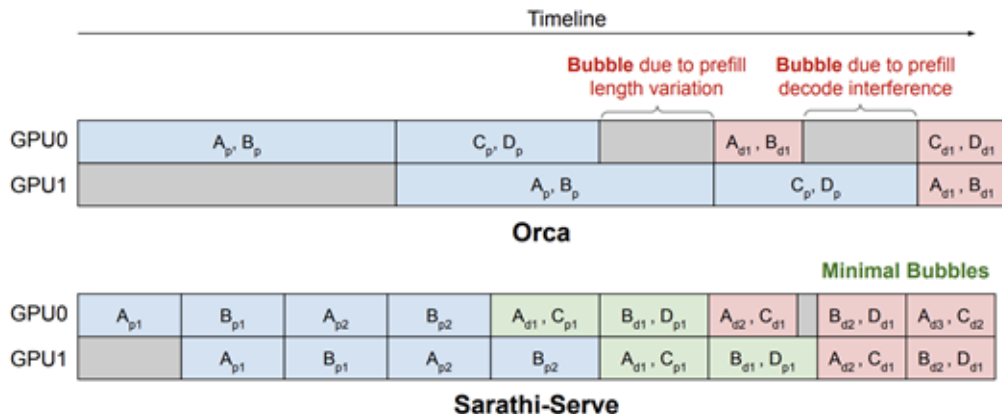
Improvements to Parallelism: Pipeline

Pipeline Parallelism (PP): distributes consecutive transformer layers across GPUs, allowing micro-batches of tokens to flow sequentially through the pipeline stages

- Limitation: Pipeline bubbles - idle periods where a GPU waits for activations from another stage from uneven compute durations across micro-batches

Sarathi-Serve: reduces pipeline bubbles by constructing hybrid batches (decodes and prefill chunks) with nearly uniform compute cost

- Significantly improved GPU utilization and higher serving capacity in large multi-GPU deployments

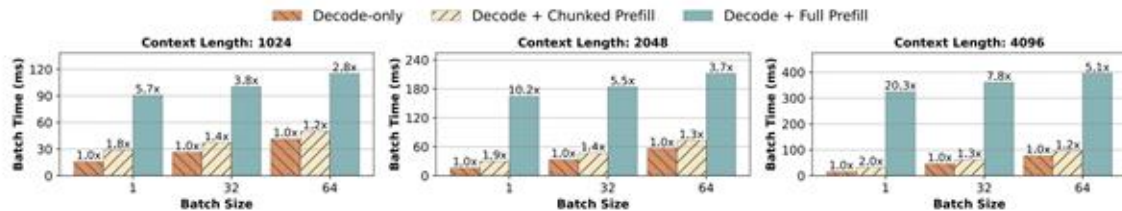


Results: Chunked Prefills Prevent Latency Explosion

Hybrid batching without chunking introduces severe latency inflation (cyan)

- observed latency increases **up to 28.3x** compared to decode-only batches (BAD)

Decode + Chunk Prefill (Beige) similar latency to Decode only (~1.2-3x)



(a) Mistral-7B on one A100s with token budget of 256.



(b) LLaMA2-70B on four A100s with token budget of 512.

Sarathi hybrid batches maintain bounded latency while enabling throughput gains

- latency overhead remains close to decode-only iterations
- effect becomes smaller with larger decode batch sizes

Results: Serving Capacity

Serving capacity measures maximum request throughput under latency SLOs

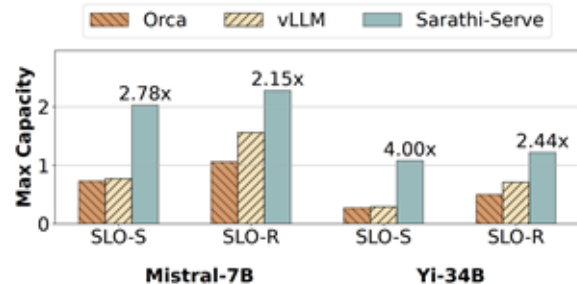
- maximum queries-per-second the system can sustain while meeting tail latency constraints

Sarathi-Serve substantially improves serving capacity across models

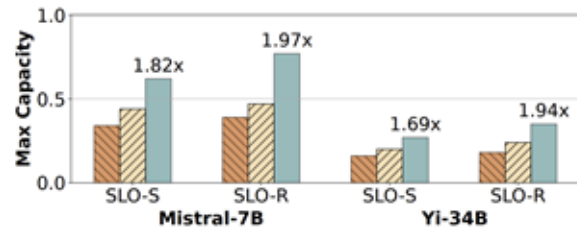
- **up to 2.7× higher capacity** for Mistral-7B on a single A100 GP
- **up to 4× higher capacity** for Yi-34B on two A100 GPUs

Throughput improvements arise from larger stable decode batch sizes

- stall-free scheduling prevents prefills from blocking decodes
- higher decode batch sizes increase GPU utilization



(a) Dataset: *openchat_sharegpt4*.



(b) Dataset: *arxiv_summarization*.

Results: Pipeline Parallelism

Pipeline parallelism divides model layers across GPUs

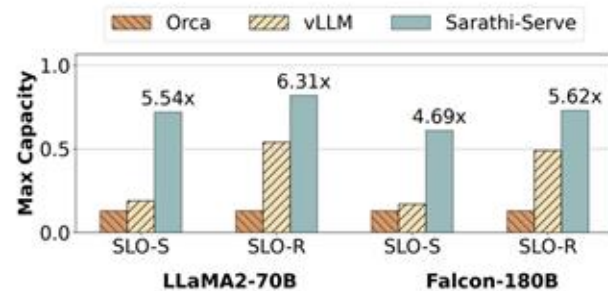
- Traditional iteration batching creates pipeline bubbles, lowering capacity

Large improvements observed for extremely large models

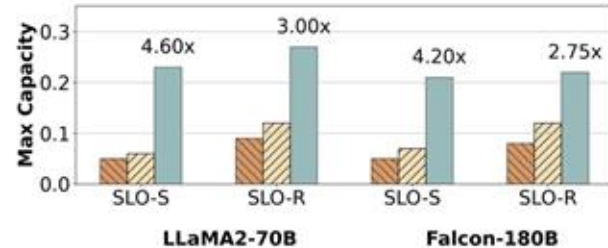
- **up to 5.6×** improvement in end-to-end serving capacity for Falcon-180B

Sarathi hybrid batches maintain uniform compute per iteration

- chunked-prefills ensure consistent compute cost across micro-batches
- balanced workloads reduce pipeline idle time



(a) Dataset: *openchat_sharegpt4*.



(b) Dataset: *arxiv_summarization*.

Conclusion

Limitations of Existing Scheduling Strategies: Force trade-off between latency and throughput

- Decode-prioritizing schedulers: preserving low token latency but wasting GPU compute when batch sizes shrink
- Prefill-prioritizing schedulers: improve throughput but interrupting ongoing decode iterations (increase TBT latency)

Sarathi-Serve Contributions

- Chunked-prefills: split large prompt prefills into smaller equal-compute segments executed across multiple iterations instead of a single long operation
- Stall-free batching: hybrid batches combine ongoing decode tokens with limited prefill chunks, bounding iteration run time
- Balanced batches: batches contain both compute-heavy prefill chunks and memory-bound decode tokens enabling better overlap of GPU compute and memory bandwidth usage

Sarathi Serve significantly increases serving capacity and eliminates of generation stalls across models and hardware configurations